

*Report
Of
Internship*

BLOG CONTENT MANAGEMENT SYSTEM (BLOG CMS)

*Submitted
To*

**School of Computer Science and Engineering
IILM University, Greater Noida, U.P.**

In partial fulfilment of the requirement of degree of

B.Tech CSE



By

**ADITYA RAJ SAXENA
(2410030189)**

Of Batch:- 2024-2028
SEMESTER- VTH

Internship/Training Organization
All India Council for Technical Education (AICTE)
and
UniConverge Technologies & upSkill Campus

Internship Completion Date: 21 August 2026
2026-27

TABLE OF CONTENTS

Chapter No.	Chapter Name	Page No.
—	Cover Page	—
1.	Candidate's Declaration	ii
2.	Acknowledgement	iii
3.	Internship Completion Certificate	iv
	List of Tables	v
	List of Figures	vi
4.	Project Description	1
4.1	Introduction	3
4.2	Organization Profile	4
4.3	Problem Statement	5
4.4	Project Objectives	6
4.5	Scope of the Project	7
4.6	Technologies and Tools Used	8
4.7	System Architecture	9
4.8	Methodology	12
4.9	Expected Outcomes	14
4.10	Certificates of Completion and Communication Proof	15
5.	Bibliography / References	19

CANDIDATE'S DECLARATION

I, ADITYA RAJ SAXENA, do hereby declare that the internship report titled “Blog Content Management System (Blog CMS)” has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering. The report presents the work, learning, implementation activities, and outcomes associated with my internship training and project work.

I further declare that the work presented in this report is based on my internship experience and the project documentation prepared during the training. References and learning resources used in the preparation of this report have been acknowledged appropriately. This report has not been submitted to any other institute or university for the fulfilment of any other degree or diploma requirement.

I shall be solely responsible for any copyright violation or misrepresentation of information contained in this report.

Signature: _____

Student Name: ADITYA RAJ SAXENA

Roll No.: 2410030189

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to IILM University, Greater Noida, and the School of Computer Science and Engineering for providing me with the opportunity to undertake internship training as part of my academic curriculum.

I am thankful to All India Council for Technical Education (AICTE), UniConverge Technologies, and upSkill Campus for the internship opportunity and the structured environment in which I was able to gain practical exposure. The internship gave me an opportunity to work on a full-stack Blog Content Management System and strengthen my understanding of frontend development, backend APIs, databases, authentication, and testing.

I am also grateful to the mentors, instructors, and support team associated with the internship for their guidance and feedback during the development process. Their support helped me work through implementation challenges related to API integration, rich-text data, authentication state, route protection, and responsive design.

Finally, I express my gratitude to my parents, teachers, friends, and everyone who supported and encouraged me throughout the internship and the preparation of this report.

Signature: _____

Student Name: ADITYA RAJ SAXENA

Roll No.: 2410030189

Date: 21st Aug, 2026

Date: 21st Aug, 2026

INTERNSHIP COMPLETION CERTIFICATE

The following official certificate is attached as documentary evidence of completion of the internship program. The certificate records the internship period as 25 July 2026 to 21 August 2026 and identifies the collaboration with All India Council for Technical Education (AICTE) and UniConverge Technologies & upSkill Campus.

Report note: The certificate itself states the training domain as “Data Science And Machine Learning”, while the four project progress reports identify the internship work as Full Stack Development on the Blog CMS. This report preserves both records and does not alter the issued certificate.



Internship Completion Certificate

LIST OF TABLES

Table No.	Title	Page No.
4.1	Scope of the Blog Content Management System	7
4.2	Technologies and Tools Used	8
4.3	Weekly Internship Progress Summary	18
4.4	User Data Model	22
4.5	BlogPost Data Model	22
4.6	Blog CMS API Endpoint Reference	24
4.7	Security and Access Controls	26
4.8	Representative Test and Validation Matrix	28
4.9	Detailed Weekly Work Log	30

LIST OF FIGURES

Figure No.	Title	Page No.
4.1	Blog CMS Use Case Diagram	2
4.2	Blog CMS System Architecture	9
4.3	Entity-Relationship Diagram of the Blog CMS	10
4.4	Blog Post CRUD Workflow	11
4.5	Blog CMS Development Methodology	12

CHAPTER 4: PROJECT DESCRIPTION

The internship project focused on the design and development of a full-stack Content Management System (CMS) for creating, editing, managing, and publishing blog content. The application combines a React-based frontend with a Node.js and Express.js backend and a MongoDB database. It also includes a drag-and-drop page builder, rich-text editing, user authentication, protected routes, and responsive interfaces.

The four weekly progress reports show a gradual development process: Week 1 concentrated on requirements, project planning, environment setup, and technology exploration; Week 2 established the React interface, Express server, MongoDB connection, and initial drag-and-drop builder; Week 3 added CRUD operations, rich-text editing, API testing, and database validation; and Week 4 focused on authentication, authorization, responsive improvements, page-builder enhancements, and end-to-end testing.

The final project structure separates the frontend and backend into clear modules and uses Git/GitHub for version control. The project is designed as a maintainable foundation for further CMS capabilities such as richer publishing workflows, media management, roles, search, analytics, and deployment.

Blog CMS - Use Case Overview

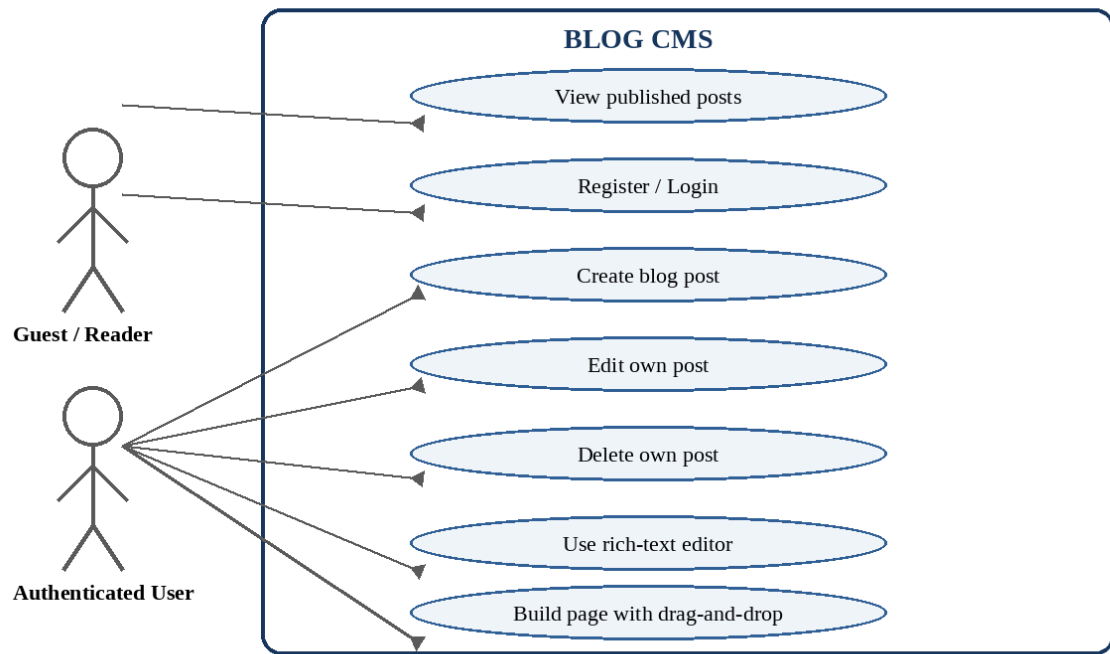


Figure 4.1: Blog CMS Use Case Diagram

4.1 Introduction

A Content Management System provides a structured way to create, organize, edit, and publish digital content without requiring every content update to be performed directly in application code. For a blog platform, a CMS should make content creation simple while also maintaining data consistency, access control, and a clear publishing workflow.

The internship project addressed this requirement through a web-based Blog CMS. The application provides a public-facing area for published posts and an authenticated management area where users can create, edit, delete, save drafts, and publish blog posts. A rich-text editor supports formatted content, while a drag-and-drop builder provides reusable page blocks such as headings, text, images, quotes, and dividers.

The frontend was developed with React.js and React Router. The backend uses Node.js and Express.js to expose REST-style endpoints, while MongoDB with Mongoose is used for persistence. Authentication is implemented using JSON Web Tokens (JWT), and passwords are protected using bcrypt-based hashing.

The project therefore provided practical exposure to the complete application flow: user interaction in the browser, frontend state management, API communication, server-side validation and authorization, database operations, and final testing.

4.2 Organization Profile

The internship documentation identifies upSkill Campus as the internship provider and UniConverge Technologies Pvt. Ltd. as its industry partner. The internship offer letter states that the candidate was appointed as a Full Stack Development Intern and that the internship was structured for a period of four to six weeks with working hours of 10:00 AM to 6:00 PM. The offer letter also identifies the internship as a remote engagement.

The completion certificate records the internship program as being conducted in collaboration with the All India Council for Technical Education (AICTE) and UniConverge Technologies & upSkill Campus. It confirms successful completion from 25 July 2026 to 21 August 2026 and carries Certificate ID USC444472TIA.

The project-specific weekly progress reports identify the working domain as Full Stack Development and the project as “Content Management System (Blog CMS).” The reports document the technical progression of the project from initial planning and setup through frontend/backend integration, CRUD functionality, rich-text editing, authentication, authorization, responsive design, and end-to-end testing.

The internship therefore served as a practical bridge between academic learning and software development practice, particularly in the areas of web application architecture, REST APIs, database integration, authentication, UI development, debugging, testing, and version control.

4.3 Problem Statement

Traditional blog publishing workflows can become difficult to maintain when content creation, page layout, data storage, authentication, and publishing are handled through separate tools or manually maintained code. A useful CMS should bring these activities together in one application while ensuring that content remains structured and access to management functions is controlled.

The primary problem addressed by this project was to develop a working Blog CMS that allows authenticated users to manage blog content through a browser-based interface while providing a public interface for published content. The system needed to support both structured data operations and flexible content formatting.

The project specifically addressed the following requirements:

- Provide user registration and login with secure password handling.
- Protect content-management operations using token-based authentication and authorization.
- Create, read, update, and delete blog posts through REST APIs.
- Support draft and published post states.
- Provide rich-text editing for headings, paragraphs, lists, and formatted content.
- Provide a drag-and-drop page builder with multiple content blocks.
- Maintain blog data using MongoDB and Mongoose validation.
- Provide responsive interfaces for desktop, tablet, and mobile screen sizes.
- Test the complete workflow and resolve integration and usability issues before final submission.

4.4 Project Objectives

The main objective of the internship project was to design and implement a functional full-stack Blog CMS while developing practical skills across the major layers of a modern web application.

1. Understand and apply a client–server architecture using React, Node.js, Express.js, and MongoDB.
2. Develop a modular React frontend with reusable navigation, dashboard, editor, and post components.
3. Implement a drag-and-drop page builder using React-DnD with heading, text, image, quote, and divider blocks.
4. Implement blog CRUD operations through backend REST endpoints.
5. Integrate a rich-text editor for formatted blog content.
6. Design MongoDB schemas for users and blog posts and apply validation to important fields.
7. Implement registration and login using JWT-based authentication and bcrypt password hashing.
8. Protect create, update, and delete operations so that only authenticated users and the appropriate author can perform restricted actions.
9. Use Postman and browser developer tools to test APIs, debug errors, and verify complete workflows.
10. Improve responsiveness and usability across different screen sizes.
11. Use Git and GitHub to maintain a structured development history and track project changes.

4.5 Scope of the Project

The scope of the project covers the complete development cycle of a small full-stack Blog CMS. It includes the user-facing interface, content-management dashboard, backend services, database models, authentication, page-building components, and testing activities.

Table 4.1: Scope of the Blog Content Management System

Area	Scope Covered
User Management	Registration, login, JWT authentication, bcrypt password hashing, authentication state.
Blog Management	Create, read, update and delete operations; draft and published states.
Content Editing	Rich-text editing and storage of formatted blog content.
Page Builder	Drag-and-drop blocks for heading, text, image, quote and divider components.
Backend	Express server, REST routes, controllers, middleware, validation and error handling.
Database	MongoDB with Mongoose models for users and blog posts.
Frontend	React, React Router, reusable components, dashboard and public pages.
Testing	Postman API testing, browser debugging, responsive checks and end-to-end workflow testing.

Future scope can include role-based administration, media uploads, categories and tags, search and filtering, comments, analytics, richer publishing workflows, automated testing, deployment, and additional security controls. These items were not treated as core deliverables of the internship project.

4.6 Technologies and Tools Used

Table 4.2: Technologies and Tools Used

Technology / Tool	Purpose in the Project
React.js	Frontend application and reusable UI components.
React Router	Client-side navigation between public and authenticated pages.
React-DnD	Drag-and-drop page-builder functionality.
React Quill / Quill	Rich-text blog content editing.
Node.js	JavaScript runtime for backend execution.
Express.js	REST API server, routes and middleware.
MongoDB	Document-oriented database for users and blog posts.
Mongoose	MongoDB object modelling, schema validation and queries.
JSON Web Tokens (JWT)	Token-based authentication for protected operations.
bcryptjs	Password hashing and password verification.
Postman	REST API testing and request/response verification.
Git & GitHub	Version control, change tracking and project management.
VS Code	Primary development and code-editing environment.

The final project files confirm a separated client/server structure. The client uses React 18.3.1 with React Router 6.26.1, React-DnD 16.0.1 and React Quill 2.0.0. The server uses Express 4.19.2, Mongoose 8.5.1, JSON Web Token 9.0.2 and bcryptjs 2.4.3. These versions are documented in the project package files.

4.7 System Architecture

The Blog CMS follows a layered client–server architecture. A user interacts with the React frontend through public pages, login/register screens, the dashboard, the editor, and the page builder. The frontend communicates with the Express backend through REST API requests. The backend applies validation, authentication, authorization, business logic, and database operations before returning responses to the client.

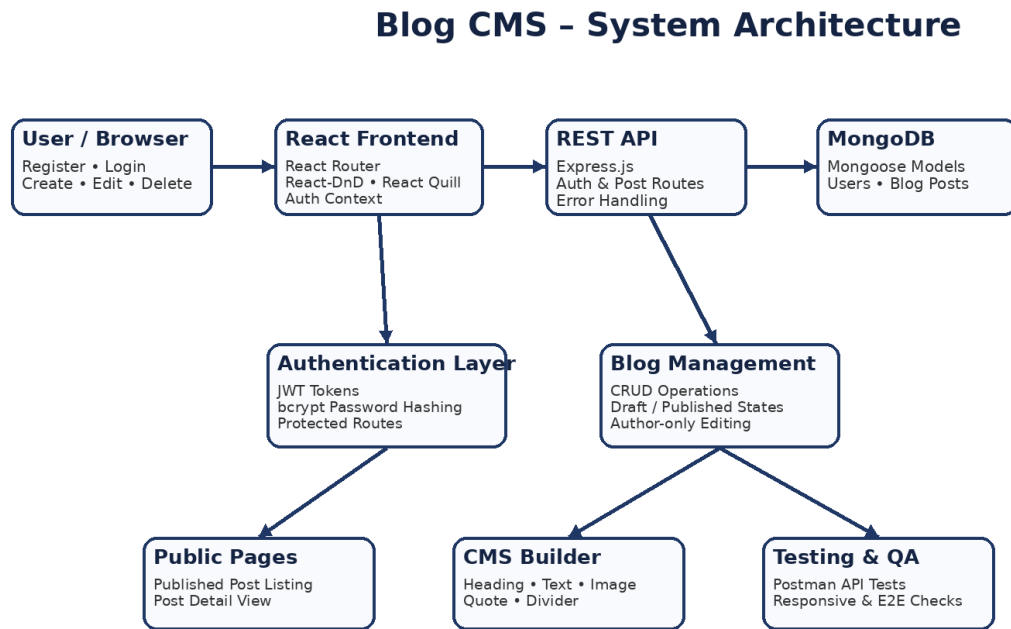


Figure 4.2: Blog CMS System Architecture

The main backend data models are User and BlogPost. User records contain name, email, password and created date. BlogPost records contain title, content, author, publish status, created date and updated date. The author relationship allows the server to restrict update and delete operations to the owner of a post.

Blog CMS - ER Diagram

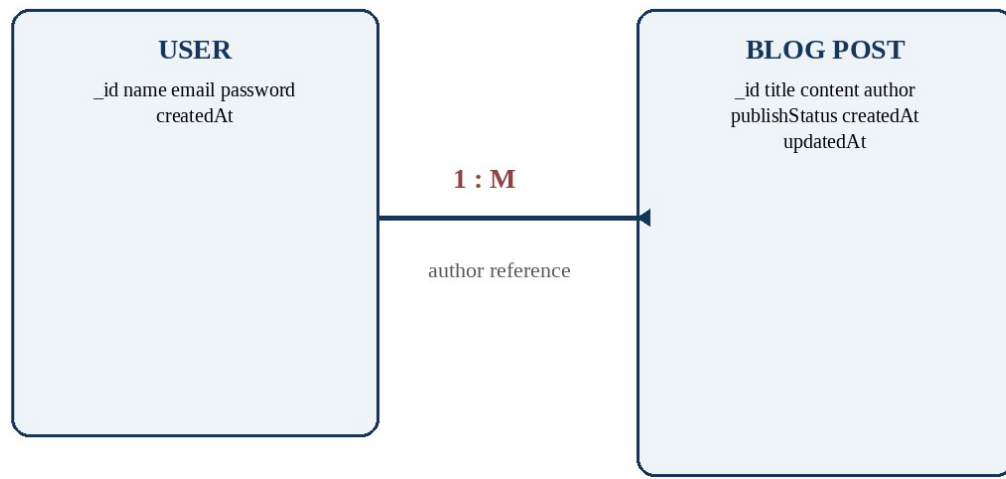
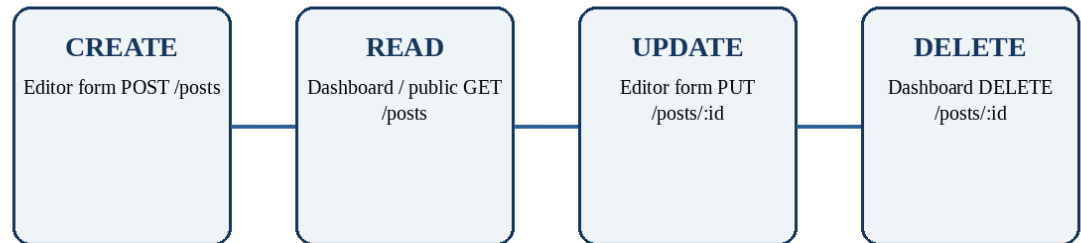


Figure 4.3: Entity-Relationship Diagram of the Blog CMS

The application exposes public endpoints for published content and authentication endpoints for registration and login. Authenticated endpoints support retrieving the current user's posts and creating, updating, or deleting posts. Protected routes use JWT verification middleware, while centralized error handling is applied at the server level.

Blog CMS - Blog CRUD Workflow



All four operations are protected by authentication, server-side authorization and database validation.

Figure 4.4: Blog Post CRUD Workflow

4.8 Methodology

The development methodology followed a progressive, feature-by-feature approach. The project was divided into manageable stages so that the frontend, backend, database, authentication, content features, and testing could be implemented and verified incrementally.

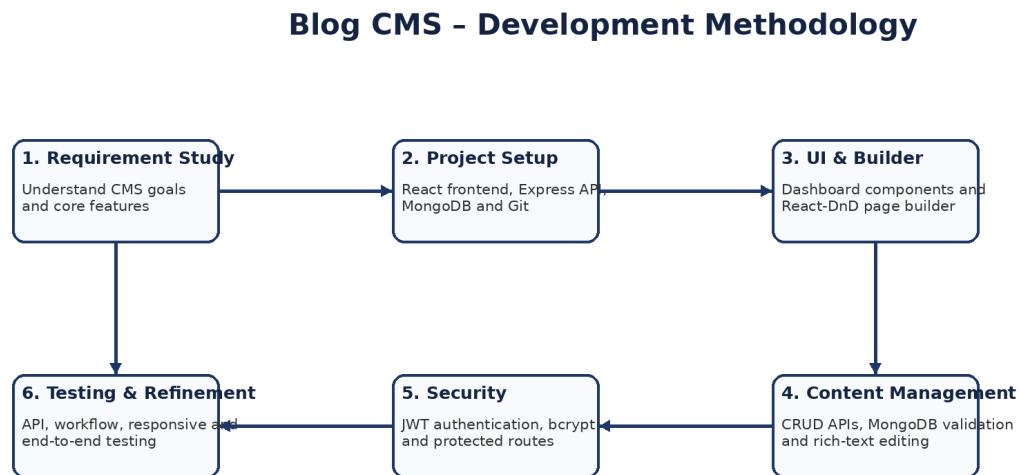


Figure 4.5: Blog CMS Development Methodology

Week 1 – Planning and Setup

Studied the CMS requirements, explored reference CMS concepts, selected the React–Node/Express–MongoDB stack, configured the development environment, initialized Git/GitHub, and explored React-DnD.

Week 2 – Core Architecture

Built the initial React dashboard, reusable Navbar/Sidebar/Workspace components, started the drag-and-drop builder, configured Express routes and controllers, connected MongoDB, and designed the blog-post schema.

Week 3 – Content Management

Implemented CRUD operations, integrated a rich-text editor, connected forms to APIs, added validation, refined MongoDB handling, improved the dashboard, and tested API behaviour using Postman.

Week 4 – Security and Finalization

Implemented JWT login/registration, bcrypt password hashing, protected routes, author-only operations, additional page-builder blocks, responsive improvements, end-to-end workflow testing, bug fixing, and code cleanup.

4.9 Expected Outcomes

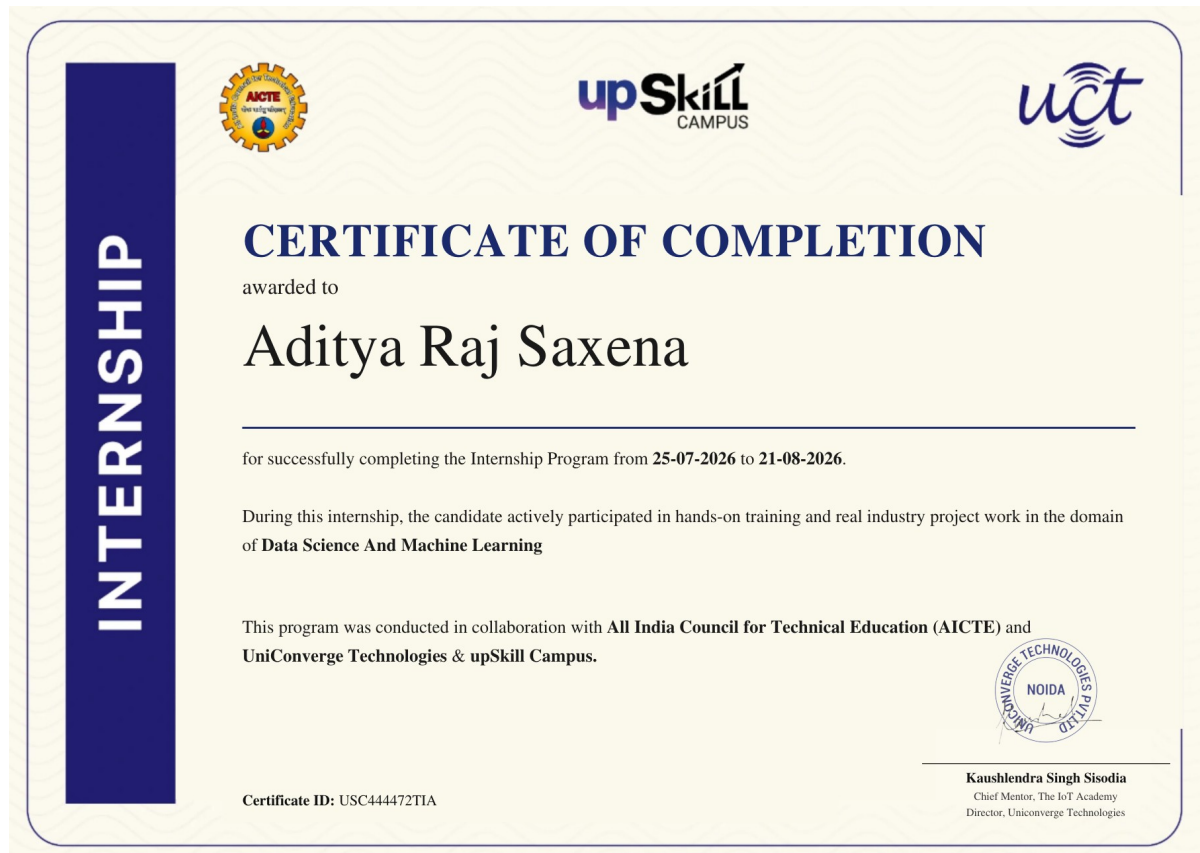
The internship was expected to provide practical exposure to full-stack web development and to result in a working CMS prototype. The completed project and weekly progress documentation demonstrate the following outcomes:

- A functional React-based frontend with public pages and an authenticated content-management dashboard.
- A structured Express.js backend with authentication and blog-management routes.
- Successful MongoDB integration through Mongoose models and validation.
- Working blog CRUD functionality with draft and published states.
- Rich-text editing for formatted blog content.
- A reusable drag-and-drop page builder with multiple content blocks.
- JWT-based authentication with bcrypt password hashing and protected operations.
- Author-level authorization for updating and deleting blog posts.
- Responsive layouts tested across smaller and larger screen sizes.
- Practical experience in debugging API errors, route configuration, authentication state, and UI behaviour.
- Improved understanding of modular project structure, version control, API testing, and end-to-end verification.

The completed project also provides a foundation for future enhancements such as deployment, richer media handling, role-based permissions, content categorization, search, analytics, automated tests, and production-level security hardening.

4.10 Certificates of Completion and Communication Proof

This section contains the documentary evidence and progress summary associated with the internship. The university format requires the completion certificate and relevant communication or supporting proof to be attached. The official completion certificate is reproduced below.



Internship Completion Certificate

The certificate confirms completion of the Internship Program from 25 July 2026 to 21 August 2026 and identifies the collaboration with AICTE and UniConverge Technologies & upSkill Campus. The certificate also carries Certificate ID USC444472TIA.

Supporting Communication / Offer Letter

The internship offer letter is included below as supporting documentation. The official completion certificate is used as the controlling record for the internship period stated in this report.

OFFICIAL	INTERNSHIP	DATE	RECONCILIATION
As per the official Internship Completion Certificate used for this report:	25 July 2026 to		
21	August		2026.

The completion certificate is the controlling document for the internship period stated in this report. The offer letter shown below is reproduced as originally issued and has not been altered.



C-56/11, Ground floor, Near IT Stellar Park, Sector-62, Noida, U.P. - 201309
Email: support@upskillcampus.com, Ph. No. 9354068856

Internship Offer Letter

Date: **25-05-2026**

Name: **Aditya Raj Saxena**

Email: **adityarajsaxena558@gmail.com**

Location: **Remote**

Dear **Aditya Raj Saxena**,

We are pleased to inform you that **Upskill Campus** alongwith its industry Partner **UniConverge Technologies Pvt. Ltd.** has agreed to provide you internship as **Full Stack Development Intern**. Your appointment of service is based on the following terms and conditions.

1. The internship shall start from **25-05-2026**
2. **Internship:** The internship period will be 4 to 6 weeks, starting from the date of joining.
3. **Working Hours:** The internship runs from 10:00 AM to 6:00 PM.

On behalf of both the Company, we wish you every success in your position and trust that our relationship will be long and mutually rewarding.

Sincerely

For Upskill Campus and UniConverge Technologies Pvt. Ltd.



Prabha Singh
HR Manager

Weekly Internship Progress Summary

The following summary consolidates the four weekly progress reports submitted during the internship. The reports identify the project as a Content Management System (Blog CMS) under the Full Stack Development domain.

Table 4.3: Weekly Internship Progress Summary

Week	Major Work Completed	Key Learning / Challenge
Week 1	Requirements, project planning, environment setup, React exploration, Git/GitHub setup, React-DnD research.	Technology selection, modular planning, development setup and drag-and-drop concepts.
Week 2	React dashboard, reusable components, initial builder, Express server, MongoDB setup, schema design and API connectivity.	Component communication, backend integration, API testing and project organization.
Week 3	Blog CRUD, rich-text editor, API routes, MongoDB validation, dashboard refinement and Postman testing.	API error handling, formatted content storage and debugging CRUD routes.
Week 4	JWT login/registration, bcrypt hashing, protected routes, builder enhancements, responsive design and end-to-end testing.	Authentication state, authorization middleware, responsive breakpoints and final QA.

By the end of Week 4, the project had reached a feature-complete state for the internship build cycle, with authentication, authorization, page-builder improvements, responsive layouts, and end-to-end workflow testing completed. The remaining focus documented in the Week 4 report was final documentation, demonstration, and a final clean-environment review.

CHAPTER 5: BIBLIOGRAPHY / REFERENCES

The following resources and project records were referred to during the internship and in the preparation of this report:

1. React.js Official Documentation – component development, hooks, project structure and frontend concepts.
2. React Router Documentation – client-side routing and protected navigation patterns.
3. React-DnD Documentation – drag-and-drop interfaces and draggable/drop-target concepts.
4. React Quill / Quill Documentation – rich-text editing and formatted content handling.
5. Express.js Documentation – backend routing, middleware and REST API development.
6. Node.js Documentation – JavaScript runtime and server-side application development.
7. MongoDB Documentation – document database concepts and data management.
8. Mongoose Documentation – schema definition, validation, models and database queries.
9. JSON Web Token Documentation – token-based authentication concepts.
10. bcrypt / bcryptjs Documentation – password hashing and verification.
11. MDN Web Docs – web standards, JavaScript, CSS, responsive design and browser development tools.
12. Postman API Testing Tool – REST endpoint testing and debugging.
13. Git and GitHub documentation – version control and project collaboration.
14. Internship Offer Letter issued by upSkill Campus / UniConverge Technologies Pvt. Ltd., dated 25 May 2026.
15. Internship Completion Certificate issued to Aditya Raj Saxena, confirming completion on 21 August 2026.
16. Four Weekly Progress Reports for the Blog CMS Full Stack Development Internship.
17. IILM University Internship Report Format updated 26–27.

18. Project source files and README for the Blog CMS application submitted with the internship documentation.

APPENDIX A: FUNCTIONAL MODULES AND USER WORKFLOW

The Blog CMS is organized around a set of connected modules that collectively support content creation, content management, publication and public consumption. The implementation separates the client interface from the server-side application logic and uses a persistent MongoDB data layer.

Module	Primary Responsibility	Important Elements
Authentication	Create accounts and establish signed sessions.	Registration, login, JWT token, bcrypt password hash, two-hour token lifetime.
Public Blog	Display published content to visitors.	Post listing, post detail, author name, publication status filtering.
Dashboard	Provide the authenticated management workspace.	My posts, create, edit, delete and draft/publish controls.
Rich-Text Editor	Allow structured and formatted post composition.	Quill editor, headings, paragraphs, lists and rich content storage.
Page Builder	Assemble reusable content blocks through drag and drop.	Heading, text, image, quote and divider blocks.
Backend API	Validate requests and perform business operations.	Express routes, controllers, middleware and centralized error handling.
Database Layer	Persist application records.	User and BlogPost Mongoose models, validation and timestamps.

A typical authenticated workflow begins with registration or login. Once the server validates the credentials, a JWT is returned to the client and the authentication state is maintained through a React context provider. The user can then enter the dashboard, create a post, save it as a draft or publish it, and later update or delete the post as its author.

For visitors, the public workflow is intentionally simpler. Published posts are returned by the public posts endpoint, while unpublished content remains inaccessible to ordinary visitors. The system therefore separates content management from content consumption and keeps the publication state explicit.

The page builder adds a second content-creation path. Instead of composing a page only through a single text field, the user can arrange modular blocks. This provides a visual structure that can be extended later with additional block types or richer layout controls.

From an architectural perspective, each module has a clear responsibility. This separation reduces the impact of changes: authentication logic remains in authentication controllers

and middleware, database rules remain in models, and rendering concerns remain in the React client.

APPENDIX B: DATABASE DESIGN AND DATA FLOW

The project uses MongoDB as the persistent database and Mongoose as the object modelling layer. Two primary collections are represented in the source code: User and BlogPost. The design keeps user credentials separate from blog content and links each blog post to its author through a MongoDB ObjectId reference.

Table 4.4: User Data Model

Field	User Model	Purpose / Validation
name	String	Required; trimmed; minimum 2 and maximum 60 characters.
email	String	Required, unique, lowercase and trimmed.
password	String	Required; minimum 6 characters; stored as a bcrypt hash.
createdDate	Date	Automatically assigned when the user record is created.

Table 4.5: BlogPost Data Model

Field	BlogPost Model	Purpose / Validation
title	String	Required, trimmed, minimum 3 and maximum 160 characters.
content	String	Required; stores the rich-text content.
author	ObjectId	Required reference to the User model.
publishStatus	String	Enum limited to draft or published; default is draft.
createdDate	Date	Automatically assigned during creation.
updatedAt	Date	Updated through a pre-save hook whenever the post is saved.

The application data flow can be summarized as: browser action -> React component/state -> HTTP request -> Express route -> middleware/controller -> Mongoose model -> MongoDB -> controller response -> client state update -> rendered interface. This sequence is used for both authentication and blog-management operations.

A particularly important relationship is the author link. When a post is created, the backend assigns `req.user._id` to the author field. Later, update and delete operations compare the stored author identifier with the authenticated user identifier. This provides ownership-level authorization without exposing database credentials to the client.

The database layer also supports reliable ordering. Public posts are sorted using `createdDate` in descending order, while the authenticated user's own posts are returned using `updatedAt` in descending order. This makes recent content changes visible in the management workspace.

APPENDIX C: API DESIGN AND ENDPOINT REFERENCE

The backend exposes REST-style endpoints under the /api prefix. Authentication routes are grouped under /api/auth and content routes under /api/posts. The server also exposes a lightweight /api/health endpoint for confirming that the application is responding.

Table 4.6: Blog CMS API Endpoint Reference

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Create a user account after validating name, email and password.
POST	/api/auth/login	Public	Verify credentials and return a signed JWT plus basic user details.
GET	/api/posts	Public	List published blog posts ordered by newest creation date.
GET	/api/posts/:id	Public / Owner	Return a published post, or an unpublished post to its owner.
GET	/api/posts/mine	Authenticated	Return posts belonging to the authenticated user.
POST	/api/posts	Authenticated	Create a new draft or published post.
PUT	/api/posts/:id	Author only	Update title, content and publication status.
DELETE	/api/posts/:id	Author only	Delete a blog post.
GET	/api/health	Public	Return a simple health status response.

Request validation

Registration and login reject incomplete requests. Post creation and update also require non-empty title and content. The backend normalizes email addresses to lowercase and trims text fields before storing them.

Authentication header

Protected requests are expected to carry an Authorization header in the form Bearer <token>. Middleware verifies the token signature and loads the user record before the protected controller executes.

Publication logic

The create and update controllers explicitly map the publication state to either published or draft. Any value other than published falls back to draft, preventing unsupported status values from reaching the model.

Error handling

The server applies centralized error middleware after routes. Mongoose validation errors are mapped to a client-visible 400 response, while unhandled failures return a 500 response with a generic server-error message.

The endpoint design keeps responsibilities clear and makes the backend testable with tools such as Postman. It also establishes a foundation for future additions including categories, comments, search, pagination, file uploads and role-based administration.

APPENDIX D: AUTHENTICATION, AUTHORIZATION AND SECURITY

Security was a major focus during the final phase of the internship. The application does not store plain-text passwords and does not rely only on frontend checks for protected actions. Instead, the server independently validates authentication and author ownership before allowing restricted operations.

Table 4.7: Security and Access Controls

Control	Implementation	Purpose
Password hashing	bcryptjs with a work factor of 12	Protect stored credentials against direct disclosure.
JWT authentication	jsonwebtoken with a 2-hour expiry	Provide a signed, time-limited client session.
Protected routes	Express middleware checks Bearer token	Block unauthenticated access to management endpoints.
User lookup	Authenticated token maps to User record	Ensure the session still corresponds to an existing account.
Author authorization	Post author id compared to req.user id	Prevent one authenticated user from modifying another user's post.
Input validation	Required fields, string limits and status enum	Reduce invalid or malformed data at the model/API boundary.
Environment secrets	dotenv; .env excluded by .gitignore	Keep MongoDB credentials and JWT secret outside committed source.

The authentication controller performs account creation by validating required fields, normalizing the email address, checking for duplicates, hashing the password and creating the user document. It then returns a signed token along with non-sensitive user details. During login, the password is compared against the stored hash before a new token is issued.

The authorization layer is equally important. A valid JWT does not automatically grant access to every record. When a user attempts to update or delete a post, the backend first checks whether the target post exists and then verifies that its author matches the authenticated user. This implements author-level access control at the server boundary.

The project documentation also instructs developers not to commit .env files, database credentials, JWT secrets or node_modules. This separation between configuration secrets and source code is a basic but important practice for collaborative development and repository hygiene.

Security is not treated as complete merely because authentication works. Production deployment would still require HTTPS, stricter CORS settings, rate limiting, secure cookie/session strategies where appropriate, stronger validation for HTML content, logging and monitoring, automated security testing, and careful secret rotation. These are identified as future hardening areas rather than claiming that the internship prototype is production-hardened.

APPENDIX E: TESTING, VALIDATION AND TROUBLESHOOTING

Testing was carried out incrementally throughout development and became more systematic during the final week. The weekly reports describe the use of Postman, browser developer tools, responsive browser mode, and end-to-end workflow checks.

Table 4.8: Representative Test and Validation Matrix

Test Area	Representative Check	Expected Result
Registration	Valid name, email and password	Account created and token returned.
Duplicate account	Register with existing email	Request rejected with conflict message.
Login	Valid credentials	Authenticated user and JWT returned.
Invalid login	Wrong password	Authentication rejected.
Post creation	Authenticated user submits title/content	Post stored as draft or published.
Protected route	Unauthenticated create/update/delete request	Request rejected as unauthorized.
Author restriction	User B attempts to modify User A post	Request rejected as forbidden.
CRUD workflow	Create, edit, retrieve and delete post	State changes persist in database.
Responsive UI	Desktop and smaller viewport checks	Layout remains usable without major overlap.
Session expiry	Expired token used in client workflow	User session becomes invalid and protected operations fail safely.

API errors

Frontend forms initially produced request and validation errors during integration. Browser developer tools and Postman were used to compare request payloads with the backend expectations and correct malformed requests.

CRUD routing issues

Some update and delete operations initially failed because route configuration and middleware sequencing required correction. Reviewing Express routing and MongoDB queries helped isolate the failures.

Authentication state

After token expiry, some React components did not immediately reflect the changed session. Authentication state was centralized through context so components could consistently read the current session status.

Responsive layout

The drag-and-drop builder and navigation elements behaved differently on smaller viewports. Responsive browser mode and actual mobile-device checks were used to identify breakpoints and adjust CSS rules.

Final cleanup

The last validation pass included cleaning console warnings, removing unused code and checking the complete login -> create/edit/delete -> logout workflow before submission.

APPENDIX F: DETAILED WEEKLY WORK LOG AND LEARNING OUTCOMES

Table 4.9: Detailed Weekly Work Log

Week	Detailed Activities	Output / Evidence
1	Requirement analysis; CMS reference study; development environment setup; Git/GitHub initialization; React project setup; React-DnD research; feature planning by frontend/backend/database modules.	Initial project foundation, technology selection and structured feature plan.
2	Dashboard layout; Navbar, Sidebar and Workspace components; initial drag-and-drop prototype; Express server; route/controller/model folders; MongoDB connection; initial blog schema; regular Git commits.	Working client-server foundation and database connectivity.
3	Blog CRUD operations; frontend forms connected to APIs; rich-text editor; API route development; MongoDB validation; dashboard refinements; Postman verification; debugging update/delete flows.	Core content management functionality completed.
4	JWT registration/login; bcrypt hashing; protected routes; author-level authorization; additional builder blocks; responsive layout fixes; session-expiry correction; end-to-end workflow testing; code cleanup.	Feature-complete internship build ready for final submission.

The internship produced technical learning in several connected areas. On the frontend, the work strengthened understanding of reusable React components, routing, context-based state and responsive CSS. On the backend, it provided practical experience with Express middleware, REST endpoints, controllers, validation and centralized error handling. Database work developed familiarity with Mongoose schemas, references, validation rules and query behaviour.

The security work provided a clearer understanding of the difference between authentication and authorization. Authentication establishes who the user is; authorization determines what that authenticated user is allowed to do. Implementing both helped connect classroom concepts with real application behaviour.

The project also strengthened problem-solving habits. Rather than treating errors as isolated failures, the development process involved tracing requests from the browser to the API and then into the database. This layered approach made it easier to identify whether a problem belonged to the UI, payload, route, controller, middleware or persistence layer.

Finally, the internship reinforced the importance of incremental development and documentation. Each weekly milestone built upon the previous one, while Git history, weekly reports, API tests and the final project structure created an evidence trail of the implementation process. These practices are directly relevant to future software projects, team development and technical evaluation.

Overall, the Blog CMS internship project provided a practical end-to-end development experience. It moved from requirements and setup to implementation, security, debugging, testing and final readiness, while giving the student a reusable foundation for future full-stack projects.

END OF REPORT